# Building Agentic Workflows

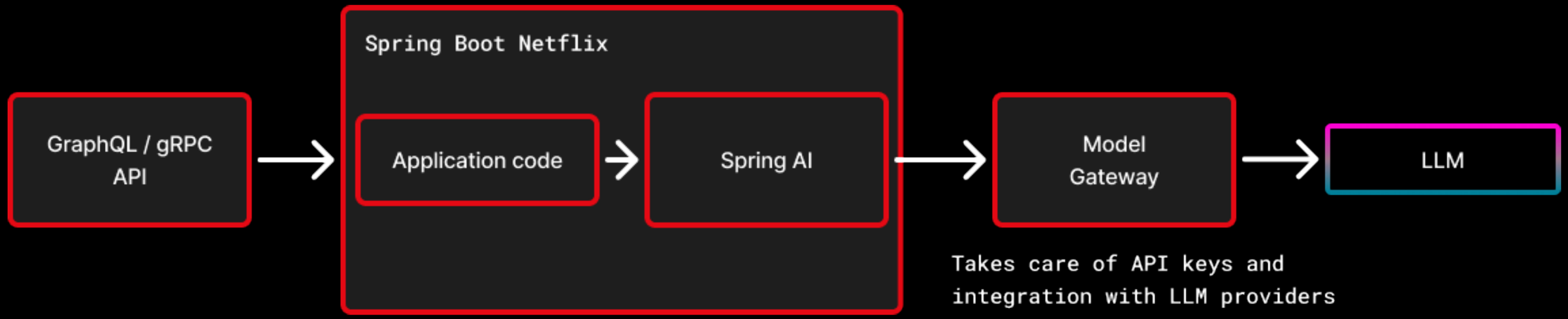
- Code to orchestrate the workflow
- LLM interaction
- Tool calls
- MCP
- Memory
- Tests/Evals



# Spring AI
# Basics

2

# Spring AI with Spring Boot Netflix



# Interacting with an LLM

```
@Component
static class SimpleChatDemo {
    private final ChatClient chatClient;

    SimpleChatDemo(ChatClient chatClient) {
        this.chatClient = chatClient;
    }

    public String chat(String question) {
        return chatClient.prompt(question).call().content();
    }
}
```





# Structured output

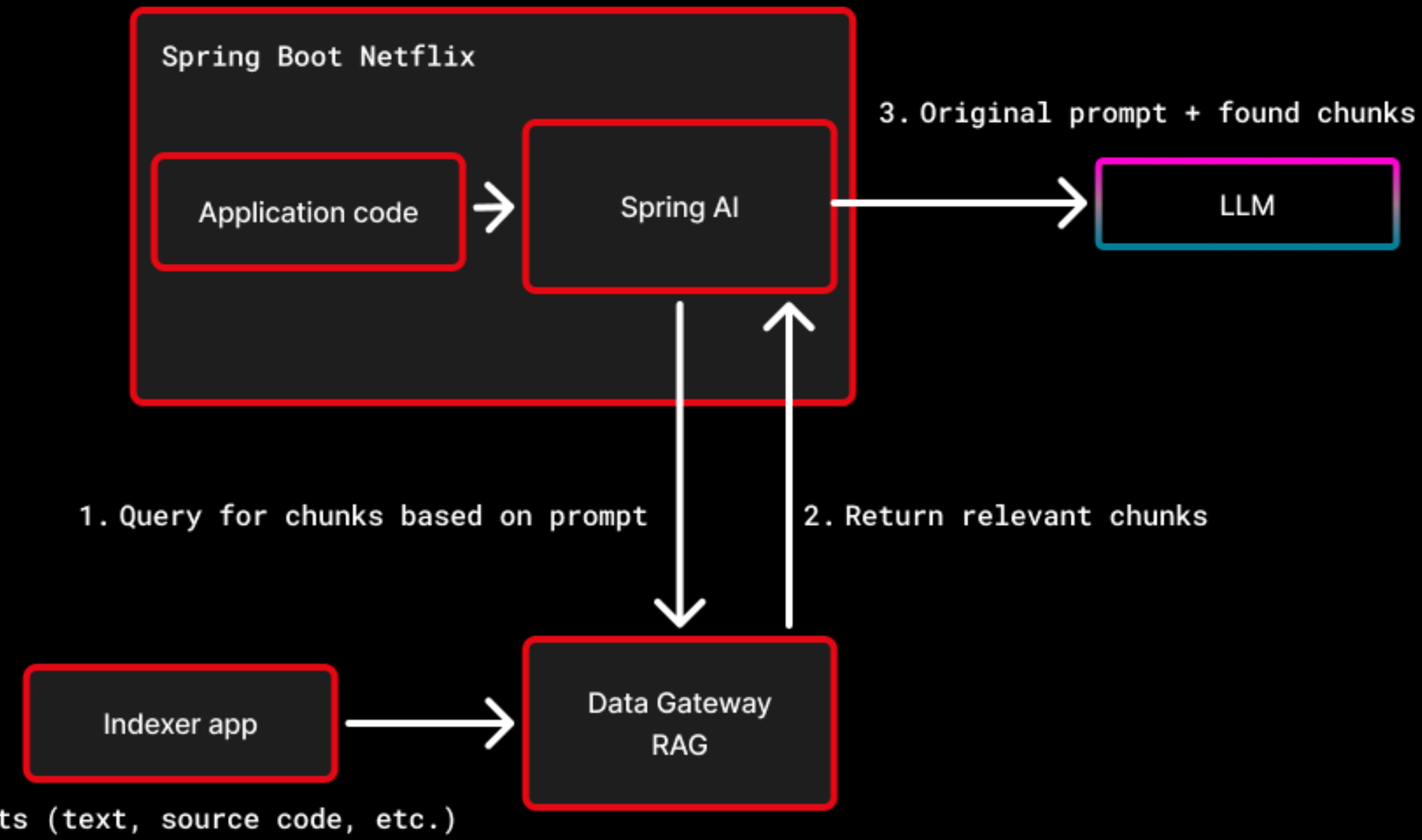
```
public List<Person> createPersonData() {
    return chatClient.prompt( content: "Create example data for 50 persons") ChatClientRequestSpec
            .call() CallResponseSpec
            .entity(new ParameterizedTypeReference<>() {});
}

public record Person(String name, String company, String email){}
```





# DGW RAG integration



# DGW RAG integration

```yaml
netflix:
  ai:
    dgwrag:
      namespaces:
        - name: javaplatform
```

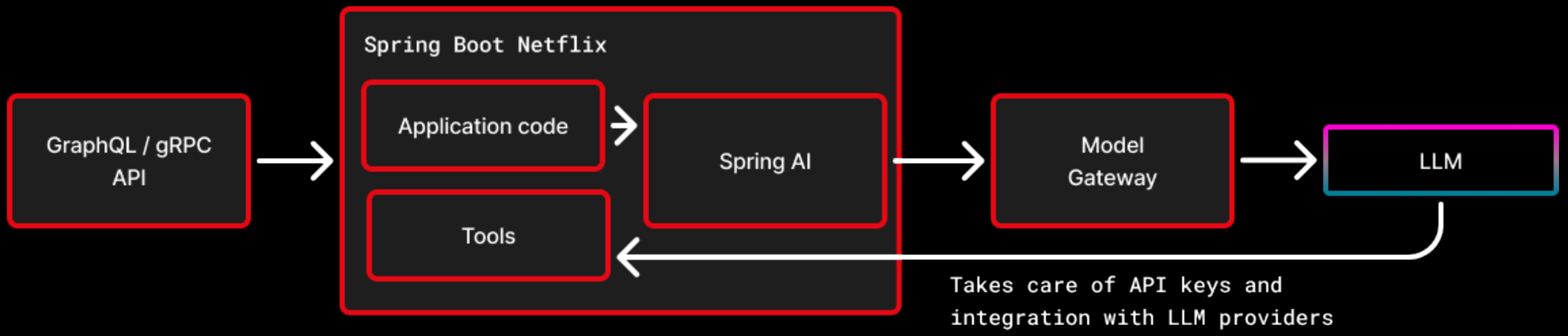
```java
private final ChatClient chatClient;
private final VectorStore vectorStore;

public RagDemo(ChatClient chatClient, @DgwRagNamespace("javaplatform") VectorStore vectorStore) {
    this.chatClient = chatClient;
    this.vectorStore = vectorStore;
}

public String sbnQuestionWithRag() {
    var qaAdvisor = QuestionAnswerAdvisor.builder(vectorStore)
            .build();

    return chatClient.prompt( content: "What is SBN?").advisors(qaAdvisor).call().content();
}
```

# Tool calling



# Tool calling

```
return chatClient.prompt(prompt) ChatClientRequestSpec
        .tools(top10)
        .call() CallResponseSpec
        .content();
```

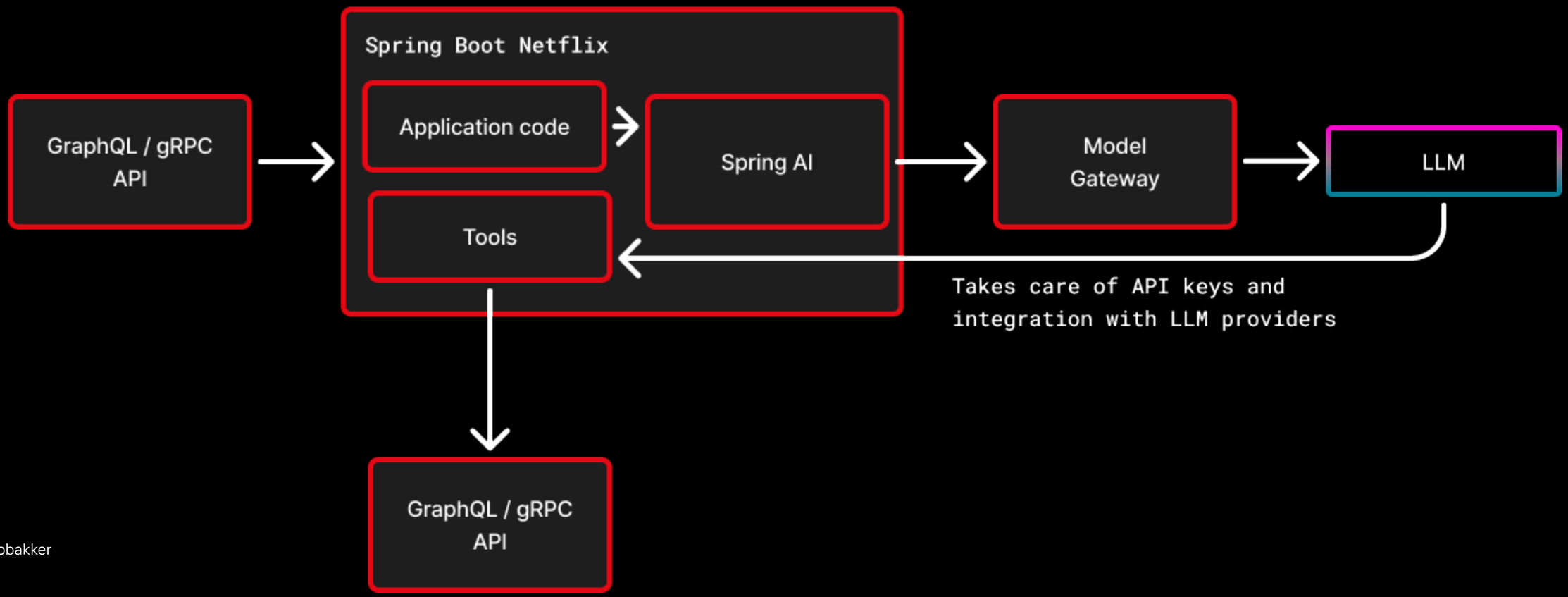
```
@Tool(description = "Use to get the current list of top 10 shows for a given genre")
public List<Show> top10Shows(@ToolParam(description = "The genre of the show") String genre) {

    LOGGER.info("Reading shows from JSON");

    var objectMapper = new ObjectMapper();
    try {
        var shows = objectMapper.readValue(resource.getContentAsByteArray(), new TypeReference<List<Show>>() {
        });

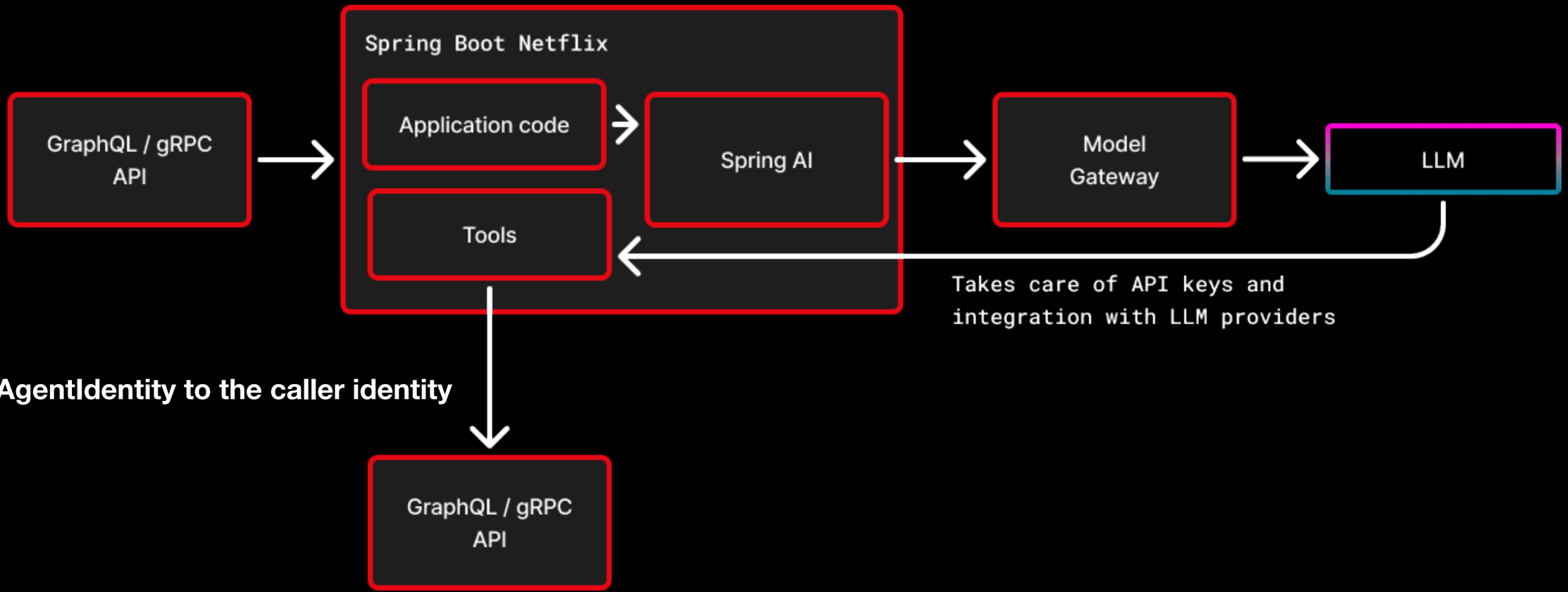

        return shows.stream().filter( Show s -> s.genre().stream().anyMatch( String g -> g.equalsIgnoreCase(genre))).toList();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

# Tool calling - call another service



# Agent Identity

- The receiving system should know it was called from an Agent



# Memory

- Models are stateless
- “Memory” implies sending message history with the next prompt

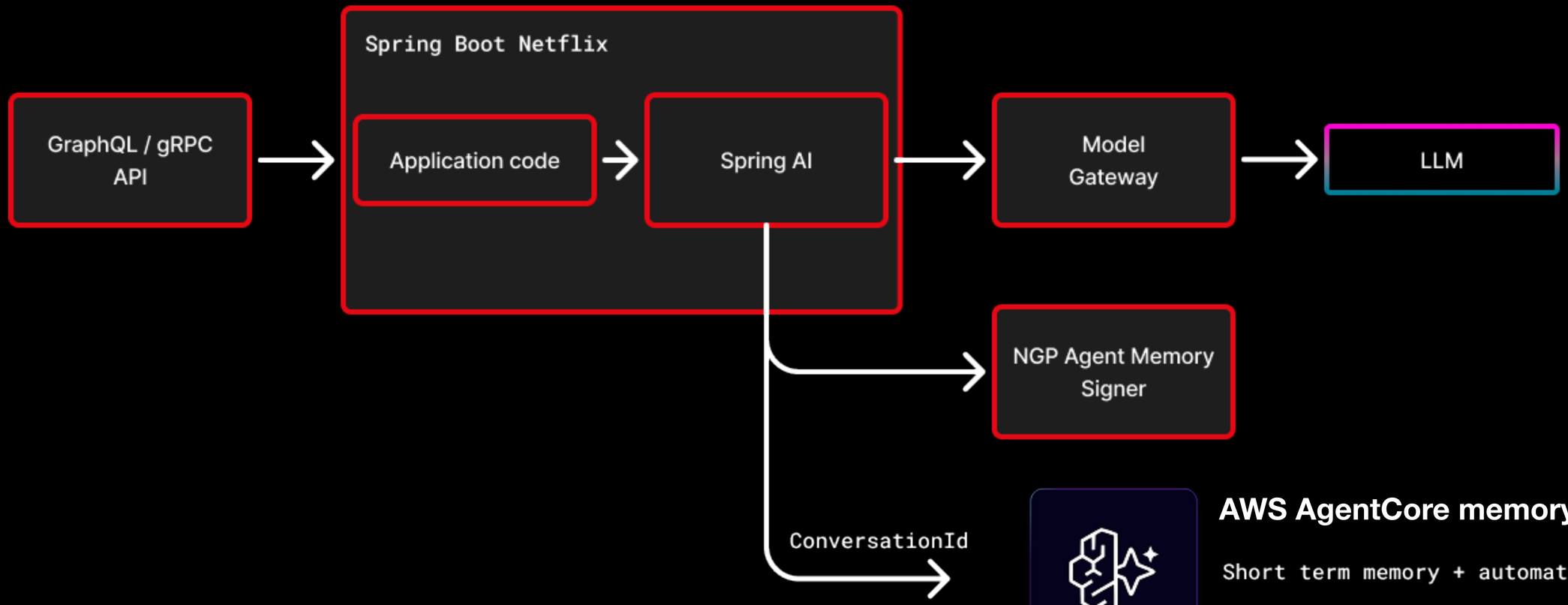


# Memory

- Short term memory: log of messages
- Long term memory: extracted key information
- Dreaming: reflect on past sessions



# Memory




**AWS AgentCore memory**

Short term memory + automated long term memory

# Multi model

```
@NetflixAiProvider(LlmProvider.ANTHROPIC) ChatClient anthropicClient,
@NetflixAiProvider(LlmProvider.OPENAI) ChatClient openaiClient,
@NetflixAiProvider(LlmProvider.GOOGLE_GENAI) ChatClient googleGenAiClient,
```

- Use @NetflixAiProvider to select the provider you need
- The model for each provider can be configured in application.yml

```
anthropic:
  chat:
    options:
      max-tokens: 64000
      model: claude-haiku-4-5
google:
  genai:
    chat:
      options:
        model: gemini-2.0-flash
openai:
  chat:
    options:
      model: gpt-5.1-chat-latest
```